RFID & OTP Based Attendance System

Smart door access and attendance logging for the E-Cell office

FSJP Project Report

Submitted by: Rushabh Makwana
PNR: MU0341120250244331
Branch: Computer Engineering | Second Year | Section B | Roll No. 16
Institute: KCCEMSR (Entrepreneurship Cell)
Academic year: 2025 - 26

Source code: github.com/buildwithcirex/Attendence-system-by-rushabh
Live portal: logs.kcecell.org

Table of Contents

1. Abstract

2. Introduction and Problem Statement

3. Objectives and Scope

4. System Overview and Architecture

5. Technology Stack

6. Hardware Design (RFID Door Controller)

7. Firmware Design

8. Cloud Backend and Database

9. Web Portal (E-Cell Attendance)

10. Security Design

11. Testing and Validation

12. Limitations and Future Scope

13. Conclusion

14. References

Appendix A - Repository Structure

Appendix B - Device API Reference

1. Abstract

Manual attendance registers in a student club office are slow, easy to forge and impossible to audit later. This project replaces them with a two-part, cloud-connected system. The first part is an **RFID door controller**: an ESP32 with an MFRC522 reader that identifies members by the UID of their RFID card, records who entered the E-Cell room, and drives a 12 V electric strike through a relay. The second part is the **E-Cell Attendance Portal**, a Next.js web application in which members check in with a six-digit one-time password (OTP) that rotates every two minutes, watch a live session timer, and write a short work summary when they check out.

Both parts share a Supabase (PostgreSQL) back end. Every card scan, door decision, check-in and check-out is stored as a timestamped row, giving administrators a searchable history of presence and hours worked. The door controller keeps working during a Wi-Fi outage by authorising cards from a locally cached allow-list and uploading its queued events once the network returns.

Keywords: RFID, MFRC522, ESP32, attendance, OTP, Next.js, Supabase, PostgreSQL, Row Level Security, IoT.

2. Introduction and Problem Statement

The Entrepreneurship Cell (E-Cell) office is shared by dozens of members who come and go through the day. Until now, presence was tracked by people signing a paper register, or not at all. This causes four concrete problems:

- **No trustworthy record.** A member can sign in for someone else, or forget to sign out, so hours worked cannot be verified.
- **No access control.** Anyone can walk into a room that contains club equipment; there is no record of who entered when.
- **No audit trail.** Paper registers get lost and cannot be searched, summarised or exported.
- **No link to work done.** Presence is recorded, but not what the member actually worked on.

The system described in this report addresses each problem. Physical entry is gated by an RFID card, and the digital attendance session can only be started by someone who is physically in the room, because the OTP is displayed on an office device and expires after two minutes.

3. Objectives and Scope

3.1 Objectives

- Identify each member automatically from an RFID card and unlock the door only for authorised, active cards.
- Log every access attempt (granted or denied, online or offline) with time, person and reason.
- Let members start an attendance session only when physically present, using a rotating OTP.
- Track session duration live and capture a work description at check-out.
- Give administrators a console to approve members, issue/replace cards, review session logs and unlock the door remotely.
- Keep the door usable when the internet is down.

3.2 Scope

The system is designed for one door and a prototype population of 50 - 100 people, on the Supabase free tier. Members must hold an institute e-mail address (@kccemsr.edu.in). The portal additionally includes task, calendar, project/budget and resource-lending modules that support day-to-day E-Cell operation; these are described briefly in Section 9.

4. System Overview and Architecture

The system has three tiers: **edge hardware** at the door, a **cloud back end** (Supabase), and a **web layer** used by members and administrators. Devices never touch the database directly; they talk to a single Supabase Edge Function over HTTPS and authenticate with a per-device secret. Browsers talk to the Next.js application, whose server-side routes use the Supabase service-role key.

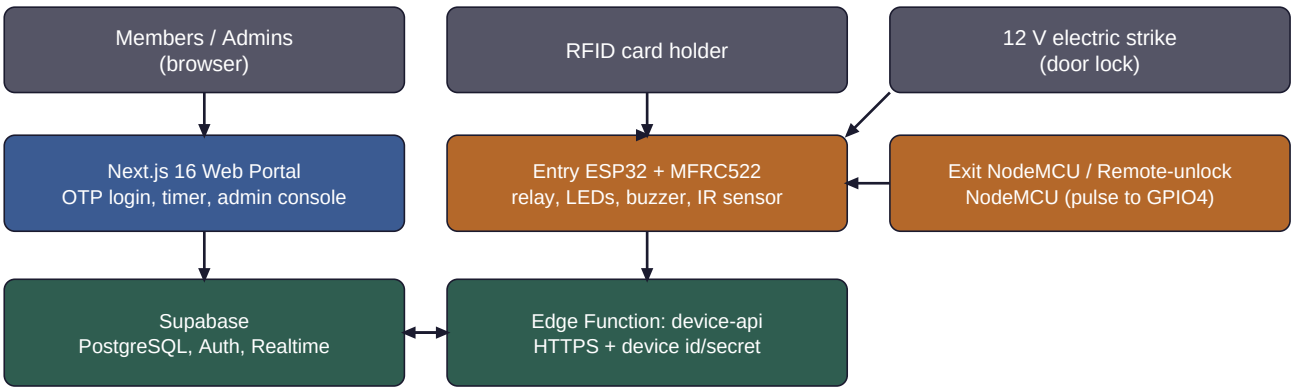


Figure 4.1 - High-level system architecture

4.1 End-to-end flow

Step	Event	What happens
1	Member taps RFID card	ESP32 reads the UID over SPI and calls *device-api* (*authorize*). The Edge Function checks card, person and door status, writes an *access_events* row and replies granted/denied.
2	Door opens	On *granted* the ESP32 energises the relay for the unlock window (5 s), shows a green LED and beeps. Denied shows red and a double chirp.
3	Member opens the portal	Enters e-mail and the 6-digit OTP shown on the office device. The server validates and consumes the OTP and creates a *sessions* row.
4	Working period	The dashboard shows a live timer against the member's chosen goal (2 / 4 / 6 / 8 h). There is no automatic logout.
5	Check-out	Member writes what they worked on; the server stores logout time, work description and duration in minutes.

5. Technology Stack

Layer	Technology	Role
Door controller	ESP32 DevKit V1, Arduino framework, PlatformIO, C++	Main entry controller: RFID, relay, sensors, state machine
Auxiliary nodes	NodeMCU (ESP8266)	Inside exit reader and remote-unlock trigger
RFID	MFRC522 13.56 MHz reader, MIFARE cards	Reads card UID over SPI
On-device storage	LittleFS	Allow-list cache and offline event queue
Back end	Supabase: PostgreSQL, Auth, Realtime, Edge Functions (Deno / TypeScript)	Data, authentication, device API, live updates
Web portal	Next.js 16, React 19, TypeScript (strict), Tailwind CSS v4, HeroUI, Framer Motion	Member and admin web application
Sessions	jose (HS256 JWT in an HTTP-only cookie)	Stateless signed login session
Admin dashboard (hardware)	Vite + TypeScript + supabase-js	Card enrolment, live access log, remote unlock
Hosting	Vercel (portal), Supabase cloud (data / functions)	Deployment

6. Hardware Design (RFID Door Controller)

The physical controller is built on perfboard and housed in a transparent enclosure fixed beside the door. The MFRC522 antenna board is taped to the glass panel of the door on a short ribbon cable, so a card can be tapped from outside the room. Figures 6.1 and 6.2 show the installed unit.



Figure 6.1 - Installed unit: MFRC522 (RFID-RC522) reader mounted on the door glass, controller enclosure below it

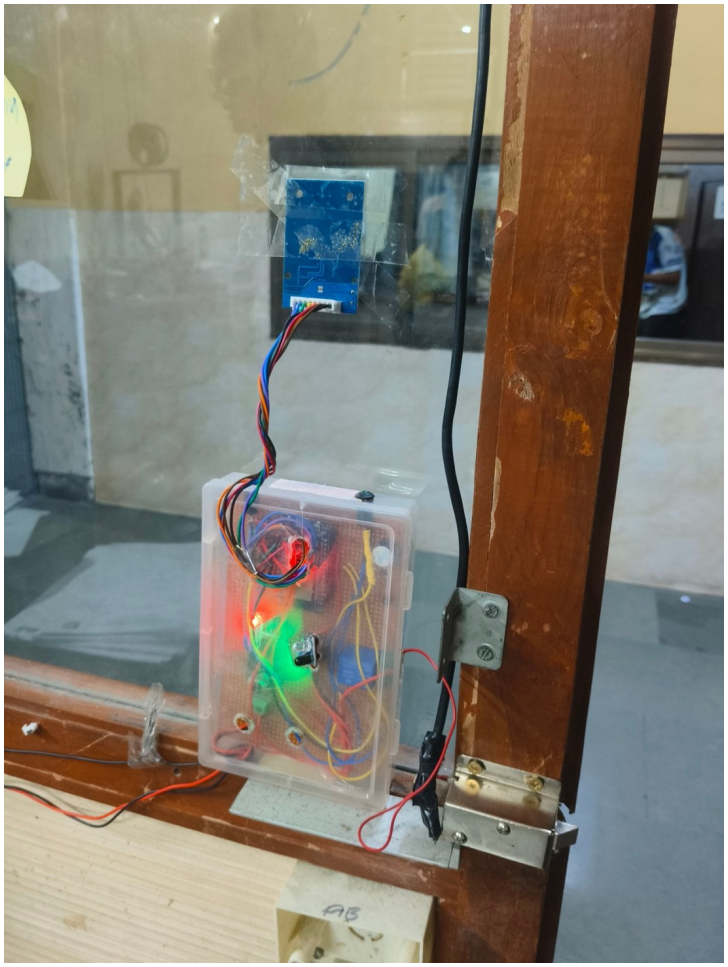


Figure 6.2 - Rear view: controller board with status LEDs, relay and IR sensor; electric strike fitted on the door frame

6.1 Bill of materials

Component	Purpose
ESP32 DevKit V1	Main controller with Wi-Fi; runs the door state machine
MFRC522 RFID module (HW-126 / RFID-RC522)	Reads 13.56 MHz card UIDs; SPI interface, 3.3 V only
5 V relay module + BC547 NPN transistor + 1 k resistor	Switches the 12 V lock from a 3.3 V GPIO safely
12 V DC fail-secure electric strike + 1N4007 flyback diode	The lock; diode clamps the inductive voltage spike
LM2596 buck converter	Steps 12 V down to 5 V for the ESP32 and relay
FC-51 IR proximity sensor	Touchless request-to-exit: wave a hand inside to unlock
Red and green LEDs (220 - 330 ohm)	Denied / granted feedback and enrolment prompts
Active buzzer (switched by BC547)	Audible feedback and held-open siren
Magnetic reed switch (optional)	Detects whether the door is physically closed
NodeMCU x 2, 1N4148 diodes x 2	Optional exit reader and remote-unlock nodes, combined by a diode-OR bus

6.2 ESP32 pin assignments

Peripheral	GPIO	Notes
MFRC522 SDA (SS)	17	SPI chip select
MFRC522 SCK / MISO / MOSI	18 / 19 / 23	SPI bus
MFRC522 RST	16	Hardware reset of RFID front end
FC-51 proximity OUT	21	Active-low, internal pull-up
Door contact (NC reed)	32	LOW = closed, HIGH = open or wire cut
Exit push button (optional)	13	Normally-closed loop
Remote exit input	4	Active-high pulse from auxiliary boards
Lock driver	27	Through 1 k to BC547 base, pulls relay IN low
Green LED / Red LED	25 / 26	Series resistors 220 - 330 ohm
Buzzer	33	Through 1 k to BC547 base

6.3 Power and noise design

- **Two power domains.** The 12 V lock rail is fused and separate from the 5 V / 3.3 V logic rail. The strike is never powered from the ESP32 or USB.
- **Flyback diode.** A 1N4007 across the lock coil absorbs the back-EMF spike when the relay opens.
- **Level shifting.** A BC547 transistor lets the 3.3 V GPIO drive the 5 V relay without exposing the ESP32 to 5 V.
- **EMI recovery.** Relay switching can upset the SPI link to the reader, so firmware re-initialises the MFRC522 350 ms after every relay action.
- **Diode-OR handshake.** Each auxiliary NodeMCU outputs a 200 ms, 3.3 V pulse through its own 1N4148 diode onto GPIO4. Either board can trigger the unlock without a bus protocol and without back-driving the other.

7. Firmware Design

The firmware (*firmware/src/main.cpp*, about 740 lines) is a cooperative, non-blocking loop; LED, buzzer and network timeouts never freeze the door logic.

7.1 Door state machine

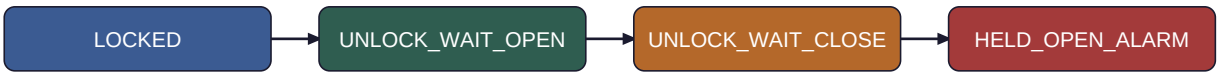


Figure 7.1 - Door states (arrows show the normal progression; any state returns to LOCKED when the door is closed)

State	Meaning	Transition
LOCKED	Relay off, lock engaged	Valid card, hand-wave or remote pulse moves to UNLOCK_WAIT_OPEN
UNLOCK_WAIT_OPEN	Relay on; 5 s window to open the door	Door opens -> UNLOCK_WAIT_CLOSE; window expires -> LOCKED

State	Meaning	Transition
UNLOCK_WAIT_CLOSE	Door is open	Closed stably for 500 ms -> LOCKED; open > 30 s -> HELD_OPEN_ALARM
HELD_OPEN_ALARM	Red LED fast blink and siren; event sent to cloud	Door finally closed -> LOCKED

If no reed switch is installed (as in the prototype), the door is assumed closed and the lock simply re-locks after the 5-second timer.

7.2 User feedback

Mode	Green	Red	Buzzer
Access granted	Solid, 1.8 s	Off	Short beep
Access denied	Off	Solid, 1.8 s	Double chirp
Enrolment: waiting for admin badge	Off	Blink 1 Hz	Off
Enrolment: waiting for new card	Blink 1 Hz	Off	Off
Card enrolled	Flash 4 Hz, 3 s	Off	Dual chirp
Door held open	Off	Fast blink	Siren 2 Hz

7.3 Offline resilience

A card tap first tries an HTTPS call with a 2-second timeout. If the network or server is unavailable, the controller falls back to a local decision:

- Every 60 s (and immediately after an enrolment) the ESP32 downloads the active-card list and writes it atomically to */allow-list.json* in LittleFS (temp file then rename), so a power cut cannot corrupt it.
- If the cache is younger than 24 hours the UID is looked up locally and the door opens for known cards; otherwise access is denied with reason *OFFLINE_CACHE_STALE*. Unknown cards are denied as *OFFLINE_UNKNOWN_CARD*.
- Each offline decision is appended to */events.ndjson* (capped at 32 KB). When Wi-Fi returns, up to 50 events at a time are uploaded with *log_batch*; the server upserts on *event_uuid*, so retries never create duplicates.

7.4 Touchless request-to-exit

From inside, an FC-51 infrared sensor detects a hand 5 - 12 cm away. The signal must stay active for 120 ms to count, and the sensor must see no hand for 700 ms before it re-arms. The event is logged as *TOUCHLESS_REX*. This avoids touching a surface and avoids needing a card to leave.

7.5 Single-reader card enrolment

A door usually has only one reader, so adding a new card must be safe from tampering. The workflow is two-step:

- An admin presses *Request badge tap* in the dashboard, creating an *enrollment_requests* row.
- The ESP32 polls, sees the request and blinks red: it needs proof of an administrator.
- An admin card is tapped; the server checks that the card is active and that its owner has role *admin*. The LED switches to green blinking.
- The new member taps the new card; *complete_enrollment* binds the UID to the person in one PostgreSQL function, and the allow-list is synced immediately.

8. Cloud Backend and Database

8.1 Device API (Supabase Edge Function)

All devices call one Deno / TypeScript Edge Function, *device-api* (about 350 lines). Every request carries *x-device-id* and *x-device-secret* headers. The function hashes the secret with SHA-256 and compares it with the stored hash using a constant-time comparison, so the plaintext secret is never stored and timing attacks are prevented. UIDs are normalised to upper-case hexadecimal before lookup. The supported actions are listed in Appendix B.

8.2 Database

The data model is held in a hosted Supabase PostgreSQL project (project ref *lmootzobnfppmjpfkgug*). Row Level Security is enabled; Realtime is enabled on the tables the dashboard watches. The tables below were derived from the application code and migration files.

Table	Used by	Key contents
members	Portal	name, e-mail, PNR, branch, year, position, role (member/admin), status (pending/approved)
otps	Portal	6-digit code, expires_at, used flag
sessions	Portal	member_id, login_time, logout_time, logout_type (manual/auto/abandoned), duration_minutes, work_description, target_minutes, otp_used
branches, years, positions	Portal	Lookup data managed by admins
tasks, calendar_events	Portal	Task assignment and shared calendar
projects, project_members, milestones, budgets, budget_items, expenses, activity_log	Portal	Project tracking with budgets
resources, resource_categories, resource_borrows	Portal	Inventory lending with borrow / return records
profiles	Hardware dashboard	Dashboard users and role
people, cards	Door system	Registered people; RFID UIDs with active flag; cascading delete
doors, devices	Door system	Door config and override mode; device id with SHA-256 secret hash
access_events	Door system	uid, decision, reason, direction, offline flag, event_uuid, device, person
enrollment_requests, remote_unlock_requests	Door system	Short-lived request/poll queues (remote unlock expires after 20 s; one active per door)
access_revision	Door system	Revision counter used for allow-list sync

9. Web Portal (E-Cell Attendance)

The portal (*logs.kcecell.org*) is a Next.js 16 application written in strict TypeScript with about 3,100 lines across its pages and API routes. The interface uses a near-black theme with hairline borders, the Gilroy typeface and spring-based motion.

9.1 Member login with a rotating OTP

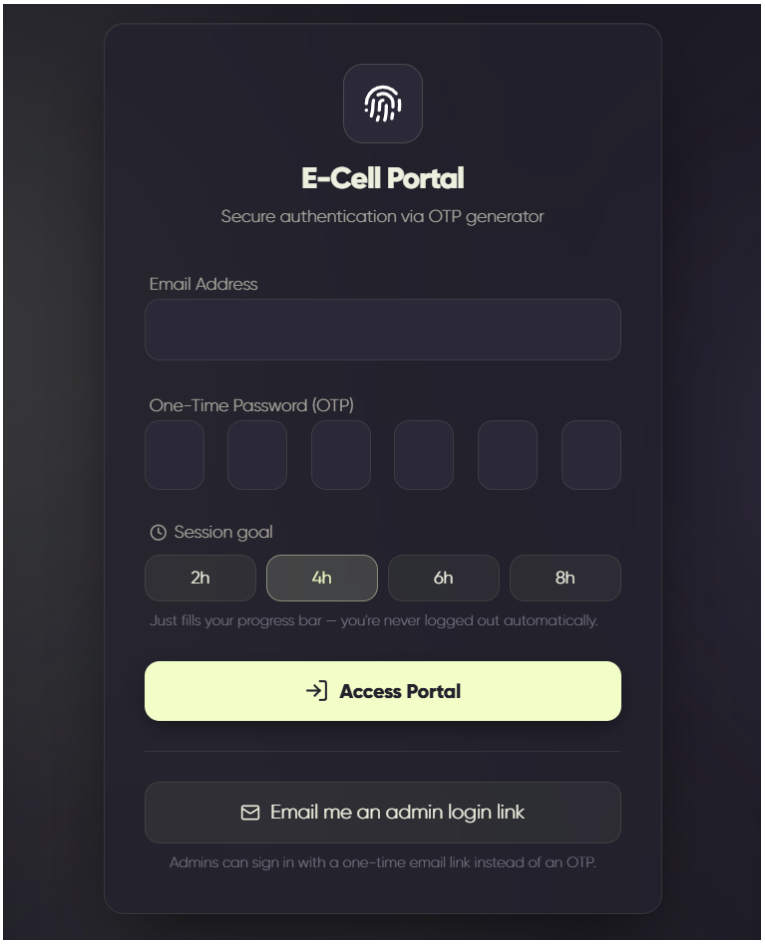


Figure 9.1 - Login screen: e-mail, 6-digit OTP, session goal, and admin magic-link option

The login route performs these checks in order: the e-mail exists in *members* (case-insensitive); it ends in *@kccemsr.edu.in*; the account is *approved*; and the OTP is unused and not expired. The OTP is then marked as used, so a code works exactly once. A new *sessions* row is created and a signed JWT is placed in an HTTP-only cookie.

The OTP itself is generated by */api/admin/otp*. If a valid unused code exists it is returned; otherwise any stale codes are invalidated and a new six-digit code valid for 120 seconds is created. The route accepts either a logged-in admin or a bearer token (*OTP_DEVICE_SECRET*, compared in constant time) so that a device in the office can display the code.

9.2 Session timer and check-out

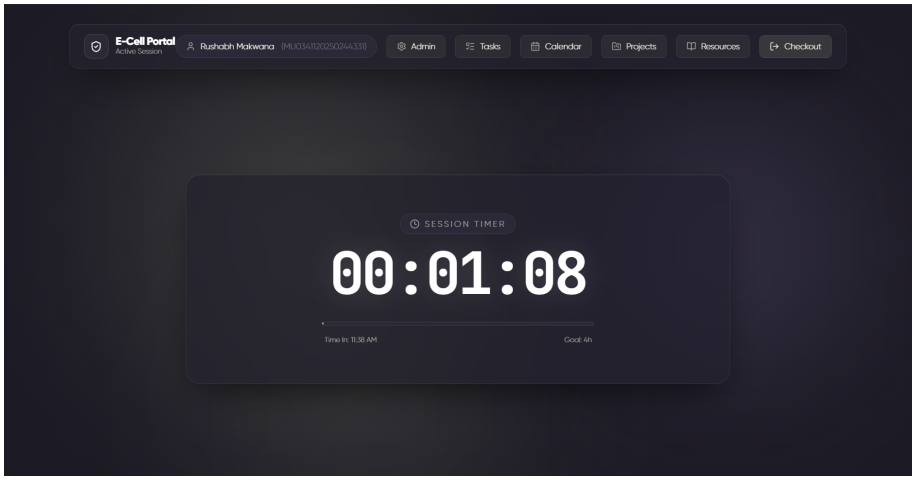


Figure 9.2 - Active session: live timer, time-in and chosen goal; navigation to tasks, calendar, projects and resources

- The member picks a goal (2, 4, 6 or 8 h; stored as *target_minutes*, validated between 30 min and 12 h). The goal only fills the progress bar; it never logs the member out.
- Logging in again on the same day resumes the open session, so the timer continues on a second device. Sessions left open from an earlier day are closed as *abandoned* with zero verified duration, which prevents multi-day timers. Day boundaries are computed in IST.
- On check-out the server computes *duration_minutes* from the signed login time and stores the work description.

9.3 Administration

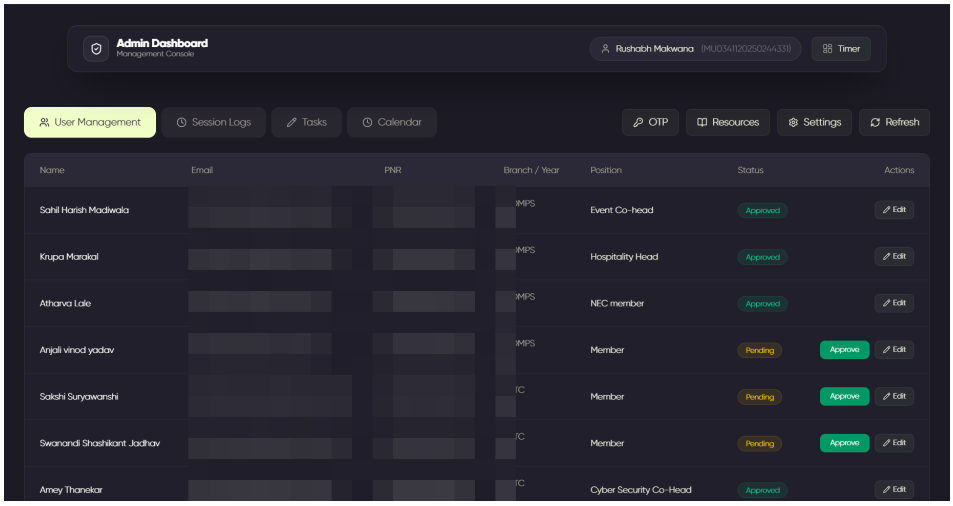


Figure 9.3 - Admin dashboard: user management with approval, plus tabs for session logs, tasks and calendar (student contact details blurred)

Admins can sign in with a one-click e-mail magic link instead of an OTP. The link is sent through Supabase Auth only to approved admins; on click the callback verifies the token (single-use, short-lived), re-checks that the member's role is *admin*, and mints the application's own JWT. From the console an admin can approve pending registrations, edit members, manage branches, years and positions, view and filter session logs, display the current OTP, assign tasks, manage the calendar and manage the resource inventory.

9.4 Supporting modules

Module	Function
Tasks	Admins assign tasks; members view and update their own
Calendar	Shared event calendar with Indian holiday data (date-holidays)
Projects	Project pages, members, milestones, activity log, budget with line items and expenses, printable budget report
Resources	Catalogue of club equipment with cover images; members borrow, return or donate items; admins track borrows

10. Security Design

Threat	Countermeasure
Cloned or guessed device identity	Per-device 64-character secret; only its SHA-256 hash is stored; constant-time comparison
Remote attendance fraud	OTP is shown only in the office, single-use, expires in 120 s
Stolen or forged session	HS256-signed JWT in an HTTP-only, SameSite=Lax cookie (Secure in production), 16 h expiry; server re-reads the signed payload and never trusts the client
Non-members signing in	Institute e-mail domain check plus admin approval before first login
Rogue card enrolment	Two-step enrolment that requires an active admin card first
Service key leakage	Service-role key only in server environment; browser and ESP32 use only the publishable key
Direct database access	Row Level Security on all tables; admin-only policies for sensitive tables
Wire cut at door contact	Normally-closed loop reads as open
Lock power faults	Fused 12 V rail, flyback diode, separate logic supply

Note. The OTP is generated with *Math.random()*, which is adequate for a short-lived code shown in a physical room but is not a cryptographically secure generator. Replacing it with *crypto.randomInt* is listed under future work.

11. Testing and Validation

The following scenarios define the acceptance tests for the system. Each lists the expected behaviour taken from the implementation; the final column is for recording the result observed at the E-Cell door.

#	Scenario	Expected behaviour	Result
1	Registered, active card tapped	Green LED, beep, relay on for 5 s, granted event logged	
2	Unknown or deactivated card	Red LED, double chirp, door stays locked, denied event logged	
3	Wi-Fi disabled, known card (cache < 24 h)	Door opens from local cache; event queued, uploaded after reconnect without duplicates	
4	Wi-Fi disabled, cache older than 24 h	Denied with OFFLINE_CACHE_STALE	
5	Hand placed in front of inside sensor	Door unlocks after 120 ms; TOUCHLESS_REX logged; no repeat until hand removed 700 ms	
6	Admin clicks remote unlock	Within ~3 s the exit/remote node pulses GPIO4 and the door unlocks	
7	Enrolment of new card	Red blink -> admin badge -> green blink -> new card -> flash and chirp; card appears in dashboard	

#	Scenario	Expected behaviour	Result
8	Login with valid OTP	Session row created, timer starts, OTP cannot be reused	
9	Login with expired or used OTP	Rejected: Invalid or expired OTP	
10	Pending or non-institute account	Rejected with pending-approval / domain message	
11	Login again same day on another device	Existing session resumed; timer continues	
12	Check-out with description	logout_time, duration_minutes and work_description saved	

12. Limitations and Future Scope

12.1 Limitations

- The RFID side and the OTP side are separate attendance sources: a card tap is logged as door access, while hours worked come from the OTP session. The two are not yet merged into one report.
- The prototype does not use a door reed switch, so held-open alarm and door-closed detection are inactive; re-lock is timer-based.
- The reflective IR proximity sensor is affected by sunlight and dark or shiny surfaces.
- Only card UIDs are used. UIDs can be cloned, so the system provides convenience and an audit trail, not high-security authentication.
- The OTP generator uses a non-cryptographic random source.

12.2 Future scope

- Correlate door events with OTP sessions to flag members who check in digitally without entering, and vice versa.
- Automatic check-in: a granted card tap could start the session without an OTP.
- Replace the FC-51 with a VL53L0X time-of-flight sensor; install the reed switch.
- Use MIFARE secure sectors or NFC with challenge-response to resist cloning.
- Monthly attendance reports exported as CSV / PDF, and e-mail or WhatsApp alerts for held-open or denied-card events.
- A mobile-friendly app or PWA for members.

13. Conclusion

The project delivers a working attendance and access-control solution for the E-Cell office. An ESP32-based RFID controller identifies members at the door and logs every decision, survives network outages through a cached allow-list and event spool, and is administered remotely from a web dashboard. A Next.js portal records attendance sessions through a rotating, presence-bound OTP, measures time worked and captures what was done. Both parts share one Supabase back end with Row Level Security, hashed device secrets and signed sessions. Together they replace the paper register with a searchable, tamper-resistant record and provide a base for richer analytics.

14. References

[1] Project source code: https://github.com/buildwithcirex/Attendence-system-by-rushabh

[2] RFID door firmware and dashboard: https://github.com/theiter8r/RFID

[3] NXP Semiconductors, MFRC522 Standard performance MIFARE and NTAG frontend, datasheet.

[4] Espressif Systems, ESP32 Series Datasheet and Arduino-ESP32 core documentation.

[5] Supabase Documentation: Auth, Row Level Security, Edge Functions, Realtime. https://supabase.com/docs

[6] Next.js Documentation, Vercel. https://nextjs.org/docs

[7] PlatformIO Documentation. https://docs.platformio.org

[8] M. Jones, J. Bradley, N. Sakimura, RFC 7519: JSON Web Token (JWT), IETF, 2015.

Appendix A - Repository Structure

Attendance portal (buildwithcirex/Attendence-system-by-rushabh)

```
src/
  app/
    login/ register/ dashboard/     member screens
    admin/ (otp, resources, settings) admin console
    projects/ resources/            project and inventory modules
    api/
      auth/      login, logout, register, admin-link, admin-callback
      admin/     users, sessions, otp, options, tasks, calendar, resources
      session/status, tasks, calendar, members, milestones, projects/[id]/...
  components/    OTPInput, LogoutModal, Navbar, Drawers, ProjectPrimitives
  utils/         session.ts (JWT), attendanceSession.ts, supabase.ts
migrations/      001_session_target_minutes.sql
supabase/email-templates/magic-link.html
```

RFID door system (theiter8r/RFID)

```
firmware/                 ESP32 entry controller (main.cpp, platformio.ini)
firmware_exit_reader/     NodeMCU inside reader
firmware_remote_unlock/   NodeMCU remote-unlock trigger
supabase/functions/device-api/index.ts   device Edge Function
supabase/migrations/      003 remote unlock, 004 any-card override, 005 enrolment admin override
web/                      Vite + TypeScript admin dashboard
queries/                  one-time SQL for admin / device setup
docs/                     WIRING.md, FLASHING.md, SUPABASE_SETUP.md, ESP32_SYSTEM_ARCHITECTURE.md
```

Appendix B - Device API Reference

All calls are HTTPS POST to */functions/v1/device-api* with headers *x-device-id*, *x-device-secret* and *apikey*.

Action	Caller	Purpose	Key fields
authorize	ESP32, exit node	Check a card and log the decision	uid, direction -> granted, reason, person, unlock_window_ms
sync	ESP32, exit node	Refresh offline allow-list	-> revision, cards[], unlock_window_ms, held_open_alarm_ms
log_batch	ESP32, exit node	Upload queued offline events	events[] -> accepted
enrollment_status	ESP32	Poll for pending enrolment	-> active, enrollment{id, person_name, status}
verify_enrollment_admin	ESP32	Validate supervisor badge	request_id, uid -> verified, admin_name
complete_enrollment	ESP32	Bind a new card to a person	request_id, uid -> enrolled, person_id
remote_unlock_status	Remote node	Poll web unlock queue (every 3 s)	-> active